Report: **Predicting Vehicle Prices Using Regression Models**

Objectives:

The primary goal of this project is to develop a robust regression model to predict used car prices for a reseller based on various listed features and specifications. In addition to predicting prices, the project focuses on identifying feature importance and mitigating overfitting through the application of regularisation techniques.

There can be several business objectives for this, such as:

- Price Prediction: Model car prices based on features like mileage, fuel type, and performance.
- Market Analysis: Explore trends and preferences in the used car market, by type, region, or other metrics.
- Feature Importance: Identify the most important factors influencing car prices (e.g., fuel type, mileage, age

This report offers a detailed analysis and predictive modelling of used car prices using Linear Regression and Regularisation techniques, specifically Ridge and Lasso. The main goal is to create an accurate price prediction model, minimize overfitting, and identify the most influential factors affecting car prices.

Dataset Overview

Context

The dataset for this task has been scraped from a German online car trading company, AutoScout. The AutoScout data is a comprehensive collection of information on various used vehicles, capturing a wide array of attributes.

Content

- The dataset is a CSV file containing 15915 records across 23 columns.
- The columns contain information on attributes such as engine specifications, vehicle weight, power, number of previous owners, vehicle condition and so on.
- There are also columns describing safety, comfort, and entertainment specifications in a bundled way. These columns contain all the extra features present as a string and might need some preprocessing.

1. Data Loading

1.1. Importing Necessary Libraries

For this project, we imported essential Python libraries for data analysis and modelling:

- pandas and numpy for data handling and numerical operations.
- matplotlib and seaborn for data visualisation.
- LinearRegression, Lasso, Ridge from the scikit-learn library to build regression models.
- train_test_split, GridSearchCV from sklearn.model_selection from splitting and model selection.
- accuracy_score, mean_squared_error, root_mean_squared_error, r2_score, mean_absolute_error, mean_absolute_percentage_error from sklearn metrics for model evaluation.

These libraries help in efficient data exploration, cleaning, and building regression models

1.2. Load the dataset

- The **Car_Price_data.csv** dataset was loaded using pandas.
- It contains **15,915 rows and 23 columns**, featuring various car attributes.
- The dataset includes both numerical and categorical features, making it suitable for regression modelling.

2. Analysis and Feature Engineering

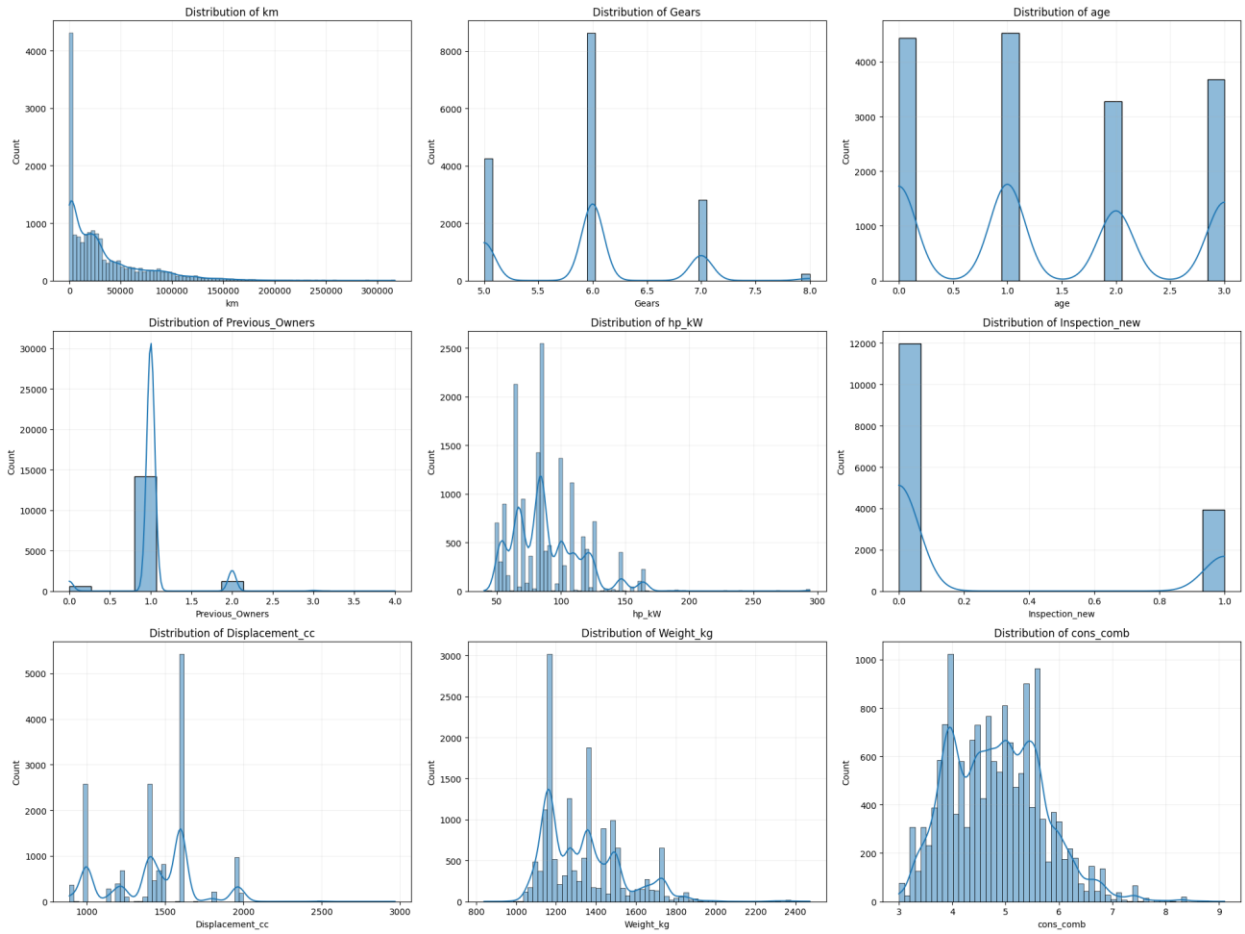
2.1. Preliminary Analysis and Frequency Distributions

2.1.1. Check and fix missing values.

- No missing values were found in the dataset.
- Target variable identified: **price**.

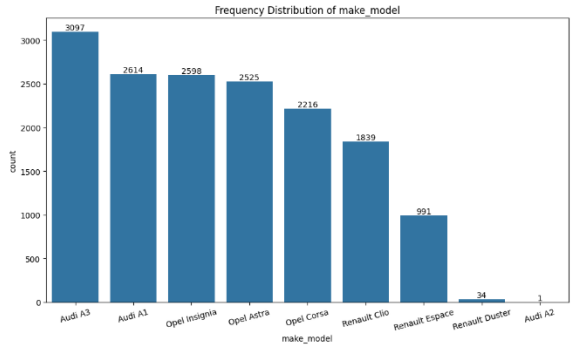
2.1.2. Identify numerical predictors and plot their frequency distributions.

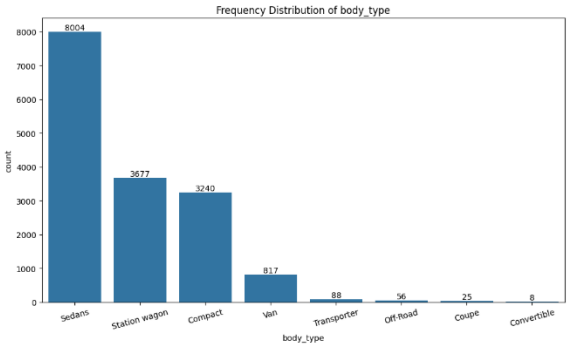
- **Numerical features (9)** include variables like 'km', 'Gears', 'age', 'Previous_Owners', 'hp_kW', 'Inspection_new', 'Displacement_cc', 'Weight_kg', 'cons_comb'.
- Most numerical features, such as km, Previous_Owners, and Inspection_new, are **right-skewed**, with values concentrated in the lower ranges.
- The features hp_kW, Displacement_cc, and Weight_kg display multiple peaks, suggesting the presence of different car categories or groups based on engine power.
- The variable cons_comb (fuel consumption) is approximately normally distributed, but exhibits a slight skew.
- Analyzing these distributions is helpful for identifying outliers and determining appropriate scaling techniques later on.

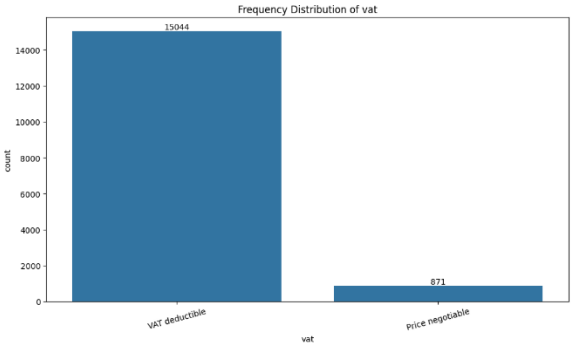


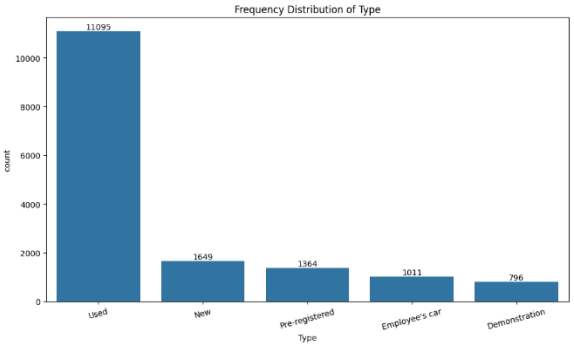
2.1.3. Identify categorical predictors and plot their frequency distributions.

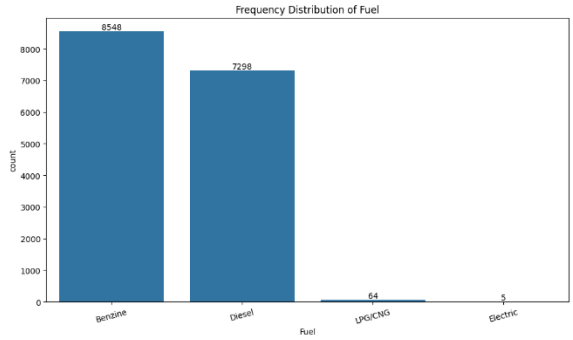
- **Categorical features (13)** include variables like 'make_model','body_type','vat','Type','Fuel','Comfort_Convenience','Entertainment_Media', 'Extras', 'Safety_Security', 'Paint_Type','Upholstery_type', 'Gearing_Type', 'Drive_chain'
- Column **Comfort_Convenience** has 6196 unique values. Column **Entertainment_Media** has 346 unique values. Column **Extras** has 659 unique values. Column **Safety_Security** has 4442 unique values. To avoid a complex graph decided to skip these columns in the frequency plot.
- These columns contain multiple bundled values, which may need further feature engineering.
- This step helps identify low-frequency categories and class imbalance, which can affect model performance.

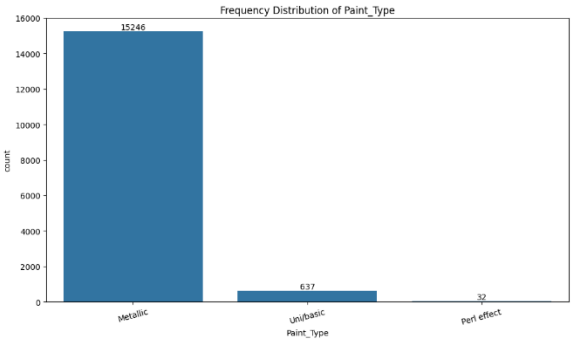

Frequency Distribution of make_model
count
make_model
Audi A3
Audi A1
Opel Insignia
Opel Astra
Opel Corsa
Renault Clio
Renault Espace
Renault Duster
Audi A2
3097
2614
2598
2525
2216
1839
991
34
1

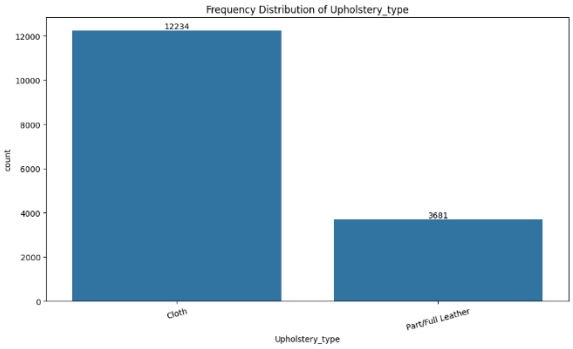

Frequency Distribution of body_type
count
body_type
Sedans
Station wagon
Compact
Van
Transporter
Off-road
Coupe
Convertible
8004
3677
3240
817
88
56
25
8

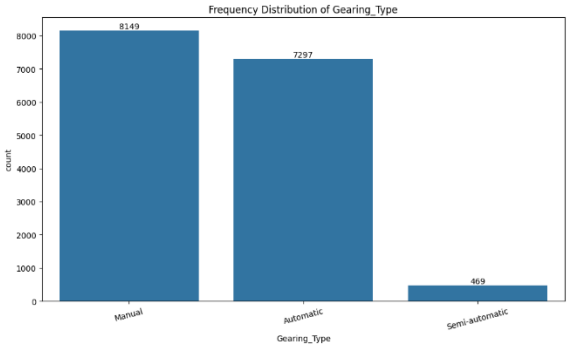

Frequency Distribution of vat
count
vat
VAT deductible
Price negotiable
15044
871

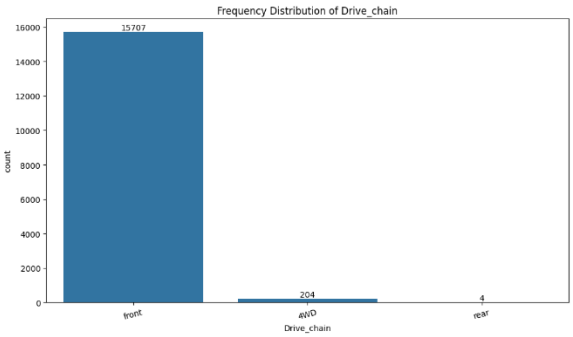

Frequency Distribution of Type
count
Type
Used
New
Pre-registered
Employee's car
Demonstration
11095
1649
1364
1011
796


Frequency Distribution of Fuel
count
Fuel
Benzine
Diesel
LPG/CNG
Electric
8548
7298
64
5


Frequency Distribution of Paint_Type
count
Paint_Type
Metallic
Uni/basic
Perl effect
15246
637
32


Frequency Distribution of Upholstery_type
count
Upholstery_type
Cloth
Part/Full Leather
12234
3681


Frequency Distribution of Gearing_Type
count
Gearing_Type
Manual
Automatic
Semi-automatic
8149
7297
469


Frequency Distribution of Drive_chain
count
Drive_chain
front
4WD
rear
15707
204
4

2.1.4. Fix columns with low frequency values and class imbalances.

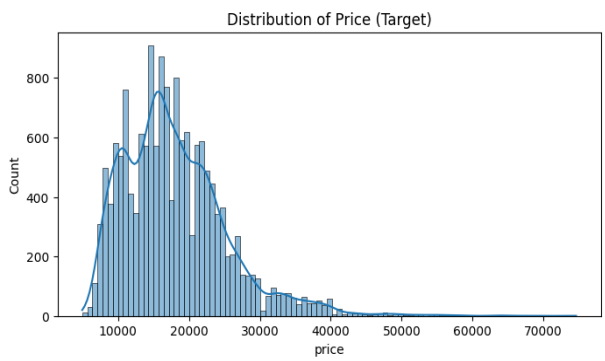
- The **Type** column was simplified by retaining the most frequently used value, 'used,' and consolidating all other low-frequency values into a single category labelled 'like new.'
- For other categorical columns, categories that represented less than **1%** of the total frequency were combined into an **'Other'** category.
- This approach reduces class imbalance and helps the model generalise better by minimising the risk of overfitting to rare categories.

2.1.5. Identify target variable and plot the frequency distributions. Apply necessary transformations.

Below is the histogram of the target variable **price** before transformation.

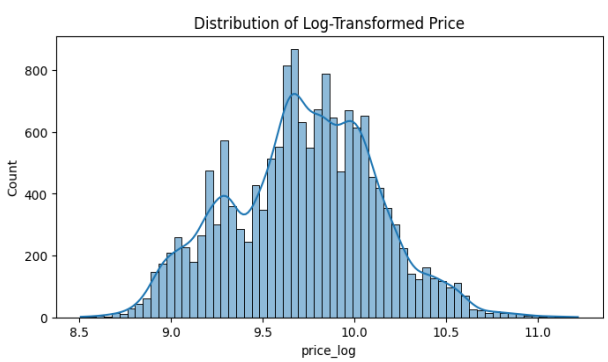
- Target Variable Distribution (Before Transformation)

  Below is the histogram of the target variable price before transformation.



  - The distribution target variable is **right-skewed** with a skew value of **1.2362**, which is greater than 1 and can be problematic for model building.
  - Most of the prices are concentrated between 10,000 and 25,000, with a long tail towards higher prices.
  - A transformation is required to normalise the distribution and reduce the impact of extreme values.

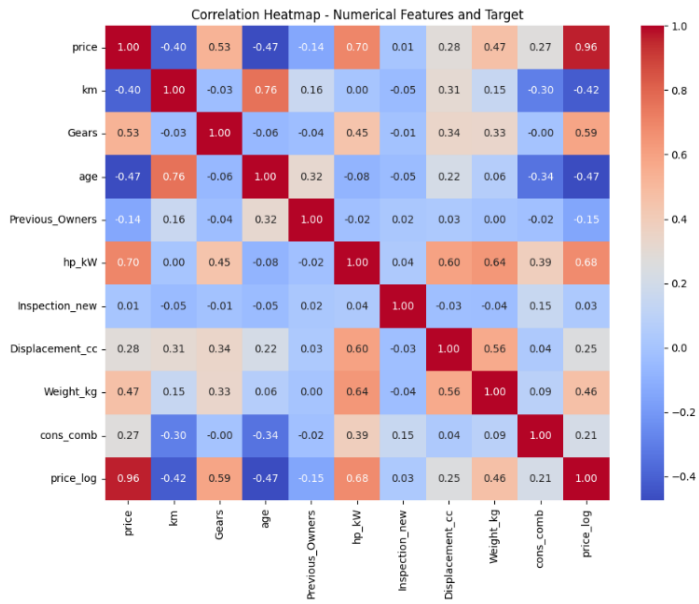
- Target Variable Transformation



  - A **log transformation (log)** was applied to the price column.
  - After transformation, the distribution became more symmetric and closer to a normal shape.
  - This will help improve model stability and reduce bias from extremely high price values.

2.2. Correlation analysis

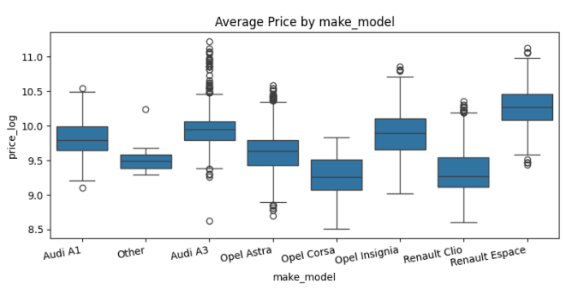
2.2.1. Plot the correlation map between features and target variable.



- **hp_kW (0.678)** and **Gears (0.588)** have the strongest positive correlation with price.
- Weight_kg, cons_comb, Displacement_cc show moderate positive correlations.
- **km (-0.419)** and **age (-0.475)** have negative correlations, indicating older and more driven cars tend to be cheaper.
- Other variables like Previous_Owners and Inspection_new have weak correlations.

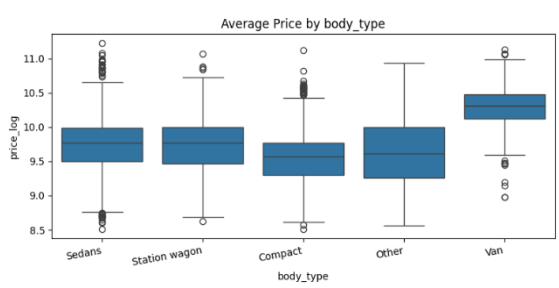
2.2.2. Analyse correlation between categorical features and target variable.

The average target value (log price) for the top categories of each categorical feature.



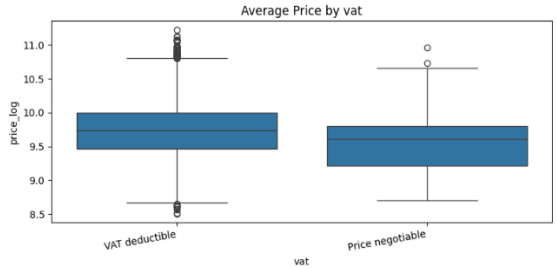
MAKE_MODEL

RENAULT ESPACE	10.271688
AUDI A3	9.928115
OPEL INSIGNIA	9.912605
AUDI A1	9.817587
OPEL ASTRA	9.625972
OTHER	9.504817
RENAULT CLIO	9.335730
OPEL CORSA	9.275962



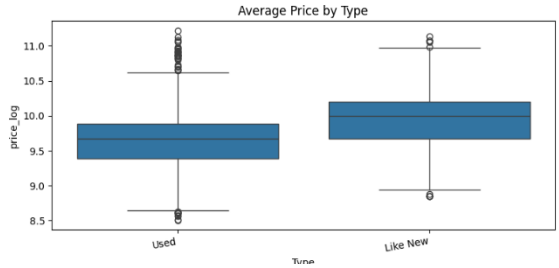
BODY_TYPE

VAN	10.294505
STATION WAGON	9.753394
SEDANS	9.716079
OTHER	9.622857
COMPACT	9.555660



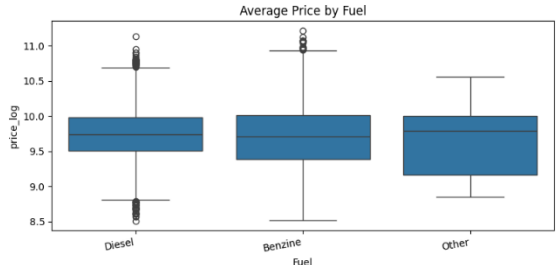
VAT

VAT DEDUCTIBLE	9.730221
PRICE NEGOTIABLE	9.556223



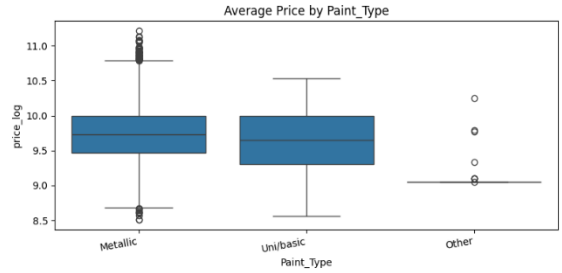
TYPE

LIKE NEW	9.928334
USED	9.630496



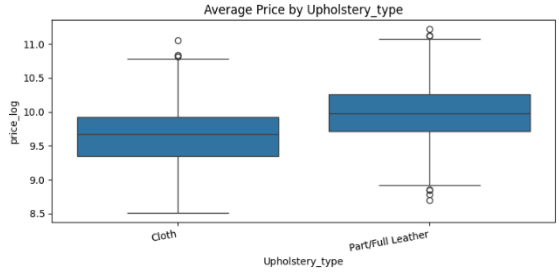
FUEL

DIESEL	9.734862
BENZINE	9.709263
OTHER	9.639389



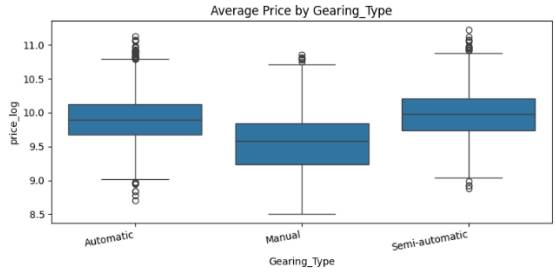
PAINT_TYPE

METALLIC	9.725340
UNI/BASIC	9.638571
OTHER	9.144455



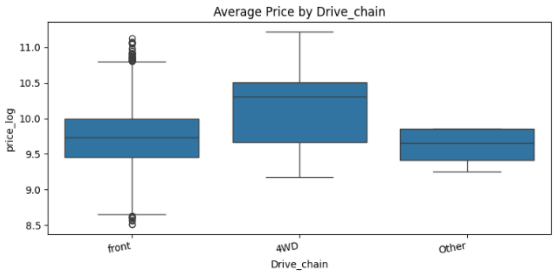
UPHOLSTERY_TYPE

PART/FULL LEATHER	9.981333
CLOTH	9.642278



GEARING_TYPE

SEMI-AUTOMATIC	9.971954
AUTOMATIC	9.906124
MANUAL	9.540199



DRIVE_CHAIN

4WD	10.133446
FRONT	9.715367
OTHER	9.606097

Top 5 Comfort_Convenience — Average Target (price_log)

Category	price_log
Air conditioning, Armrest, Automatic climate control, Cruise control, Electrically adjustable seats, Electrical side mirrors, Electric tailgate, Heated steering wheel, Hill Holder, Keyless central door lock, Leather steering wheel, Light sensor, Lumbar support, Multi-function steering wheel, Navigation system, Park Distance Control, Parking assist system camera, Parking assist system sensors front, Parking assist system sensors rear, Power windows, Rain sensor, Seat heating, Start-stop system	10.017434
Air conditioning, Automatic climate control, Cruise control, Multi-function steering wheel, Park Distance Control, Power windows	9.898697
Air conditioning, Armrest, Automatic climate control, Cruise control, Electrical side mirrors, Leather steering wheel, Light sensor, Lumbar support, Multi-function steering wheel, Navigation system, Park Distance Control, Parking assist system sensors front, Parking assist system sensors rear, Power windows, Rain sensor, Seat heating, Start-stop system	9.762334
Other	9.730333
Air conditioning, Electrical side mirrors, Hill Holder, Power windows	9.13116

Top 5 Entertainment_Media — Average Target (price_log)

Category	price_log
Bluetooth, Digital radio, Hands-free equipment, On-board computer, Radio, USB	10.018336
Bluetooth, Hands-free equipment, On-board computer, Radio	9.963639
Bluetooth, Digital radio, Hands-free equipment, MP3, On-board computer, Radio, USB	9.911254
Bluetooth, Radio	9.857451
Bluetooth, CD player, Hands-free equipment, MP3, On-board computer, Radio, Sound system, USB	9.819183

Top 5 Extras — Average Target (price_log)

Category	price_log
Alloy wheels, Sport suspension	9.942485
Alloy wheels, Roof rack, Touch screen, Voice Control	9.896199
Alloy wheels, Catalytic Converter, Voice Control	9.892244
Alloy wheels, Sport seats, Sport suspension	9.87061
Alloy wheels, Roof rack	9.836637

Top 5 Safety_Security — Average Target (price_log)

Category	price_log
ABS, Central door lock, Daytime running lights, Driver-side airbag, Electronic stability control, Immobilizer, Isofix, Passenger-side airbag, Power steering, Side airbag, Tire pressure monitoring system, Traction control, Xenon headlights	9.929989
ABS, Central door lock, Daytime running lights, Driver-side airbag, Electronic stability control, Fog lights, Immobilizer, Isofix, Passenger-side airbag, Power steering, Side airbag, Tire pressure monitoring system, Traction control, Xenon headlights	9.774858
Other	9.755703
ABS, Central door lock, Daytime running lights, Driver-side airbag, Electronic stability control, Fog lights, Immobilizer, Isofix, LED Daytime Running Lights, Passenger-side airbag, Power steering, Side airbag, Tire pressure monitoring system, Traction control	9.596737
ABS, Central door lock, Daytime running lights, Driver-side airbag, Electronic stability control, Fog lights, Immobilizer, Isofix, Passenger-side airbag, Power steering, Side airbag, Tire pressure monitoring system, Traction control	9.510322

- Some categories, such as specific make_model, Fuel, and Gearing_Type, are associated with higher average prices, while others show lower values.
- Features like Extras, Comfort_Convenience, and Safety_Security also show variations in price based on additional car features.

- This indicates that categorical attributes influence pricing significantly and should be properly encoded during model building

2.3. Outlier analysis

2.3.1. Identify potential outliers in the data.

- I developed a distribution function for outliers that calculates the count, mean, standard deviation, minimum, Q1%, Q5%, Q10%, Q25%, Q50%, Q75%, Q90%, Q95%, Q99%, and maximum value for each feature. This helps me find the best-fitting interquartile range (IQR) to identify outliers.
- After analysing it, I decided to take **(1% to 99%)** IQR.
- Applying this to all numerical columns

price:	15 outliers
km:	5 outliers
Previous_Owners:	2 outliers
hp_kW:	23 outliers
Displacement_cc:	1 outliers
Weight_kg:	49 outliers
cons_comb:	1 outliers

2.3.2. Handle the outliers suitably.

- Outliers in numerical features were treated using the IQR capping **(winsorization)** method.
- Features like km, hp_kW, and Weight_kg show a significant reduction in outliers.
- This helps improve model stability and reduces the risk of overfitting due to extreme values

2.4. Feature Engineering

2.4.1. Fix any redundant columns and create new ones if needed.

- The **"vat"** column was removed because it does not directly contribute to predicting car prices.
- I created a new feature called **"power_to_weight,"** which was calculated by dividing horsepower (hp_kW) by weight (Weight_kg). This feature represents engine performance relative to car weight. However, it displayed very high multicollinearity, indicated by a VIF value greater than 10. As a result, I decided against creating any new features.

2.4.2. Analysis and feature engineering on ['Comfort_Convenience', 'Entertainment_Media', 'Extras', 'Safety_Security'].

- The specification columns (Comfort_Convenience, Entertainment_Media, Extras, Safety_Security) contained multiple comma-separated features. Their unique values were checked to understand the type and spread of features.
- These values will add more interpretability and I depth more specific information for our prediction.
- Selects only those features that are present in **more than 10% but less than 90%** of the cars.
- This helps to ignore features that are either too rare **(not very informative)** or too common **(don't differentiate cars much).**
- Non-relevant items: ['ABS', 'Adaptive Cruise Control', 'Blind spot monitor', 'Driver drowsiness detection', 'Driver-side airbag', 'Emergency system', 'Night view assist', 'Passenger-side airbag', 'Power steering', 'Rear airbag']
- This step helped in **reducing dimensionality** and simplifying the dataset for modelling.

2.4.3. Perform feature encoding.

- I will now perform **one-hot encoding** on the categorical features to convert them into a numerical format suitable for modeling.
- This will create dummy variables for each category, and I'll drop the first dummy variable for each original column to avoid multicollinearity.

2.4.4. Split the data into training and testing sets.

- The dataset was split into training and testing sets using an **80:20** ratio.
- X_train shape: **(12732, 31)** and X_test shape: **(3183, 31)**.
- Used random state of 42 for consistent results.
- This ensures enough data for training while keeping a portion aside for unbiased model evaluation.

2.4.5. Scale the features.

- Features were scaled using **StandardScaler** on numeric columns which is best for linear regression models and regularisation.
- The scaler was it on the training set and applied to the test set.
- This standardisation improves model stability and performance.

3. Linear Regression Models

3.1. Baseline Linear Regression Model

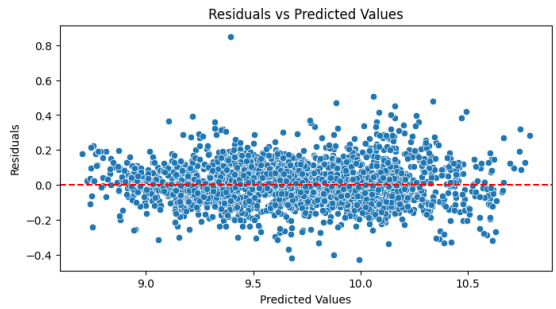
3.1.1. Build and fit a basic linear regression model. Perform evaluation using suitable metrics.

- Basic Linear Regression model was trained on scaled and encoded features.
- For evaluation I created a regression metrics function that calculate all important metrics.

Dataset	MSE	Var	RMSE	Std Dev	MAE	Mean	MAPE	R^2
Training	0.011776	0.158268	0.108519	0.397830	0.080082	9.722581	0.822428	0.925587
Testing	0.011079	0.159913	0.105259	0.399891	0.078748	9.713171	0.808364	0.930694

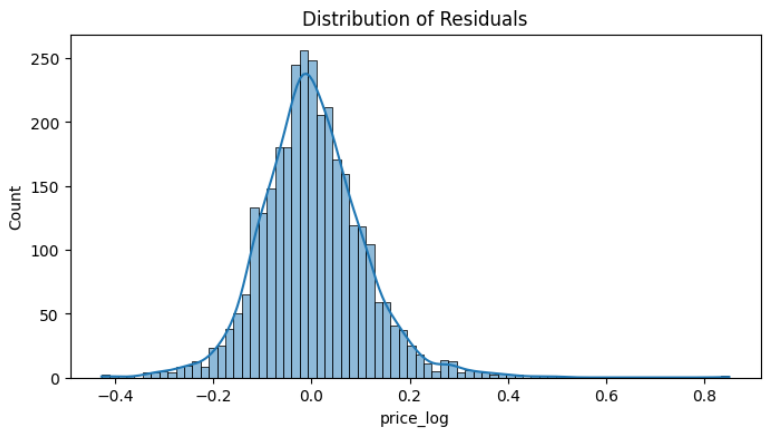
3.1.2. Analyse residuals and check other assumptions of linear regression.

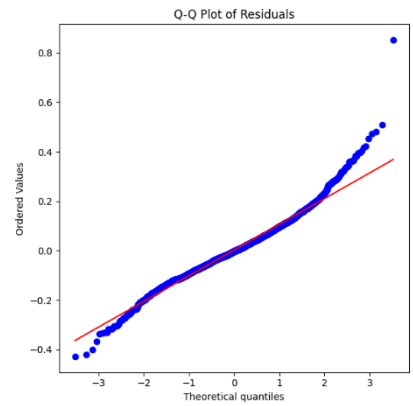
- **Linearity**: Assumed a linear relationship between the log-transformed target variable and the selected features.**(2.2.1)**



- **Independence of Errors**: Assumed that the residuals are independent.
- **Homoscedasticity**: Assumed constant variance of residuals
- Checked via scatter plot of residuals vs. predicted values.

- **Normality of Residuals**: Assumed that the residuals are normally distributed (checked via histogram and Q-Q plot).





- **No Multicollinearity**:
- High multicollinearity (VIF>10) addressed through VIF analysis and feature elimination.
- As a result, I dropped the high VIF feature "make_model_Renault Espace" with VIF=10.044.
    - mild multicorrelation (VIF between 1 and 5): found 78 features
    - moderately multicorrelation (VIF between 5 and 10): found 7 features
    - strong multicorrelation (VIF greater than 10): found 1 feature
- 78 features found in the acceptable range and 7 features found above the acceptable range that need to be treated.
- **These moderately correlated features can be addressed by regularisation.**

- **Recursive Feature Elimination (RFE)**
- Our dataset has **86 features** after working on special columns and one-hot encoding.
- I utilise the RFE technique to reduce the number of features by removing unnecessary noise from our data.
- Before doing RFE, I have built a linear model with **all features**, where I got $R^2 = 0.9302$.
- After using **40 features** (less noisy value) for building a linear model, I got $R^2 = 0.9289$.
- There is **no extreme performance difference** in model scores. From this, I conclude that those columns are just creating noise.
- This further refined the feature set, aiming to improve model generalisation and reduce noise.
- 40 is still consider as large number but better than 86 in our case. **More feature reduction is needed** for the best generalisation of our model.
- As of now, I am continuing with 40 to compare the regularisation of our model.

3.2. Ridge Regression Implementation

3.2.1. Define a list of random alpha values

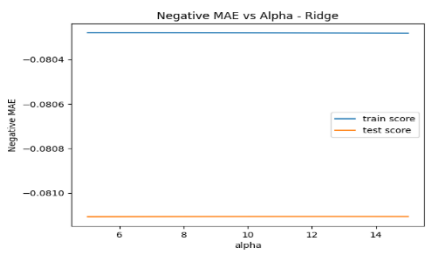
- A wide list of alpha values was defined to tune the Ridge Regression model.
- **np.linspace(0.001, 40, 1000)** → 1000 count
- The optimal alpha will be selected through tuning to balance bias–variance trade-off.

3.2.2. Apply Ridge Regularisation and find the best value of alpha from the list

- Ridge Regression was applied using multiple alpha values from the pre-defined list.
- The train–test R^2 score plot showed stable performance across most alpha values.
  - **Best alpha value: 9.6804**
  - **Best negative MAE ≈ - 0.0815**
- Regularisation helped control overfitting while maintaining high test performance.

3.2.3. Fine tune by taking a closer range of alpha based on the previous result.

- A smaller range of alpha values was used for fine-tuning alpha using "np.linspace(5, 15, 100)" selection around our previous best fit of 9.6804.



- **Best alpha (fine grid): 9.6465**
- **Best negative MAE ≈ - 0.0815.**

- This best value of alpha suggested strong shrinkage, stabilizing high VIF.
- Since we are dealing with moderately correlated features.

Dataset	MSE	Var	RMSE	Std Dev	MAE	Mean	MAPE	R^2
Training	0.012108	0.158268	0.110037	0.397830	0.081149	9.722581	0.833271	0.923490
Testing	0.011351	0.159913	0.106540	0.399891	0.079867	9.713171	0.819904	0.928996

- **Feature coefficient analysis shows**:
  - Top positive/negative coefficients are mostly related to **model_make** and **drive/gearing type**, along with **hp_KW, age, km and LED headlight**, **parking distance control** from special feature columns.
  - Least impact from columns like Inspection_new,weight_kg and some from special feature columns.

- **Top 15 and bottom 15 features by absolute coefficient:**

Top 15 features by absolute coefficient:

	feature	coefficient	abs_coef
29	make_model_Opel Corsa	-0.154408	0.154408
32	make_model_Renault Clio	-0.132681	0.132681
3	hp_kW	0.107711	0.107711
2	age	-0.098596	0.098596
28	make_model_Opel Astra	-0.092488	0.092488
0	km	-0.087585	0.087585
37	Gearing_Type_Manual	-0.058308	0.058308
30	make_model_Opel Insignia	-0.043515	0.043515
27	make_model_Audi A3	0.027847	0.027847
31	make_model_Other	-0.021619	0.021619
33	body_type_Van	0.021066	0.021066
38	Gearing_Type_Semi-automatic	0.019279	0.019279
24	Safety_Security_LED Headlights	0.018084	0.018084
35	Fuel_Diesel	0.016623	0.016623
15	Entertainment_Media_Digital radio	0.015647	0.015647

Bottom 15 features by absolute coefficient:

	feature	coefficient	abs_coef
20	Safety_Security_Central door lock with remote ...	-0.009282	0.009282
6	Comfort_Convenience_Cruise control	0.009042	0.009042
9	Comfort_Convenience_Lumbar support	-0.009014	0.009014
4	Displacement_cc	-0.008470	0.008470
5	cons_comb	0.008311	0.008311
13	Comfort_Convenience_Seat heating	0.006894	0.006894
39	Drive_chain_front	-0.006890	0.006890
10	Comfort_Convenience_Navigation system	0.006220	0.006220
23	Safety_Security_LED Daytime Running Lights	-0.006169	0.006169
22	Safety_Security_Isofix	-0.006022	0.006022
14	Comfort_Convenience_Split rear seats	0.005603	0.005603
26	Safety_Security_Xenon headlights	-0.005428	0.005428
18	Extras_Voice Control	-0.005281	0.005281
19	Safety_Security_Central door lock	-0.004978	0.004978
17	Extras_Alloy wheels	0.004973	0.004973

3.3. Lasso Regression Implementation

3.3.1. Define a list of random alpha values

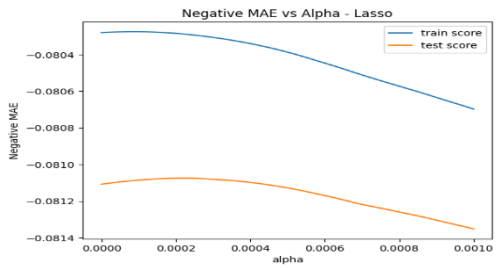
- A wide list of alpha values was defined to tune the Lasso Regression model.
- **np.linspace(0.0000001, 0.001, 1000)** → 1000 count this range allows tuning across weak to strong regularisation strengths.

3.3.2. Apply Ridge Regularisation and find the best value of alpha from the list

- Lasso can perform feature selection by shrinking some coefficients to zero.
- The optimal alpha will be selected through model evaluation and tuning.
- Lasso Regression was applied across multiple alpha values.
- The R^2 scores remained stable for smaller alphas, and performance dropped at higher alphas.
    - **Best alpha value: 0.000100190**
    - **Best negative MAE: - 0.0815**
- A very low alpha performed best, indicating minimal regularisation is sufficient for this dataset.

3.3.3. Fine tune by taking a closer range of alpha based on the previous result.

- A fine grid of smaller alpha values **"np.linspace(0.0000001, 0.0003, 100)"** selection around our previous best fit of 0.000100190.



- **Best alpha: 3e-05**
- **Best negative MAE: -0.0815.**

- Lasso performed feature selection, shrinking 5 coefficients to zero (without RFE). Since we used RFE with 40 feature lasso did not find exact zero coefficients but super close to zero.
- I tried Lasso with all **89 features**, where Lasso shrunk **5 coefficients to zero**. Which reduced 5 features.
- A very small alpha was optimal, confirming that low regularisation was sufficient.

Dataset	MSE	Var	RMSE	Std Dev	MAE	Mean	MAPE	R^2
Training	0.012108	0.158268	0.110037	0.397830	0.081148	9.722581	0.833250	0.923490
Testing	0.011351	0.159913	0.106543	0.399891	0.079871	9.713171	0.819938	0.928993

Top 15 features by absolute coefficient:

	feature	coefficient	abs_coef
67	make_model_Opel Corsa	-0.154796	0.154796
70	make_model_Renault Clio	-0.133284	0.133284
4	hp_kW	0.105393	0.105393
2	age	-0.097941	0.097941
66	make_model_Opel Astra	-0.091623	0.091623
0	km	-0.087617	0.087617
81	Gearing_Type_Manual	-0.057991	0.057991
68	make_model_Opel Insignia	-0.043182	0.043182
65	make_model_Audi A3	0.029000	0.029000
74	body_type_Van	0.021885	0.021885
69	make_model_Other	-0.020644	0.020644
82	Gearing_Type_Semi-automatic	0.018582	0.018582
58	Safety_Security_LED Headlights	0.018256	0.018256
22	Comfort_Convenience_Park Distance Control	0.015625	0.015625
76	Fuel_Diesel	0.015600	0.015600

Bottom 15 features by absolute coefficient:

	feature	coefficient	abs_coef
5	Inspection_new	-0.001454	0.001454
26	Comfort_Convenience_Rain sensor	0.001354	0.001354
71	body_type_Other	-0.001115	0.001115
12	Comfort_Convenience_Electrical side mirrors	-0.000846	0.000846
77	Fuel_Other	0.000682	0.000682
79	Paint_Type_Uni/basic	0.000535	0.000535
42	Extras_Sport seats	0.000470	0.000470
47	Safety_Security_Alarm system	0.000437	0.000437
36	Entertainment_Media_Radio	-0.000168	0.000168
18	Comfort_Convenience_Light sensor	0.000077	0.000077
7	Weight_kg	0.000000	0.000000
37	Entertainment_Media_Sound system	-0.000000	0.000000
44	Extras_Touch screen	-0.000000	0.000000
34	Entertainment_Media_MP3	0.000000	0.000000
63	Safety_Security_Traffic sign recognition	0.000000	0.000000

- Top influencing features are mainly related to **make_model, Gearing_Type, hp_KW, age, km and LED headlight and Park distance Control** from the special features column.
- Bottom features like inspection_new, weight_kg and some from special columns

3.4. Regularisation Comparison & Analysis

3.4.1. Compare the evaluation metrics for each model.

- Since there was **no overfitting** issue faced while building the model.
- All R-squared scores are similar in all three models.
- **Linear Regression** achieved the highest R^2, but may be more sensitive to noise.
- Ridge and Lasso had slightly lower scores, but better regularisation stability.
- Lasso additionally performed feature selection (5 coefficients reduced to zero).
- Ridge and Lasso results are close because multicollinearity was already low.
- Ridge performs very slightly better compared to lasso, which signifies that ridge handles multicollinearity better.

	Model	MAE	R2 Score
0	Linear Regression	0.078748	0.930694
1	Ridge Regression	0.079867	0.928996
2	Lasso Regression	0.079871	0.928993

3.4.2. Compare the coefficients for the three models. Also, visualise a few of the largest coefficients and the coefficients of features dropped by Lasso.

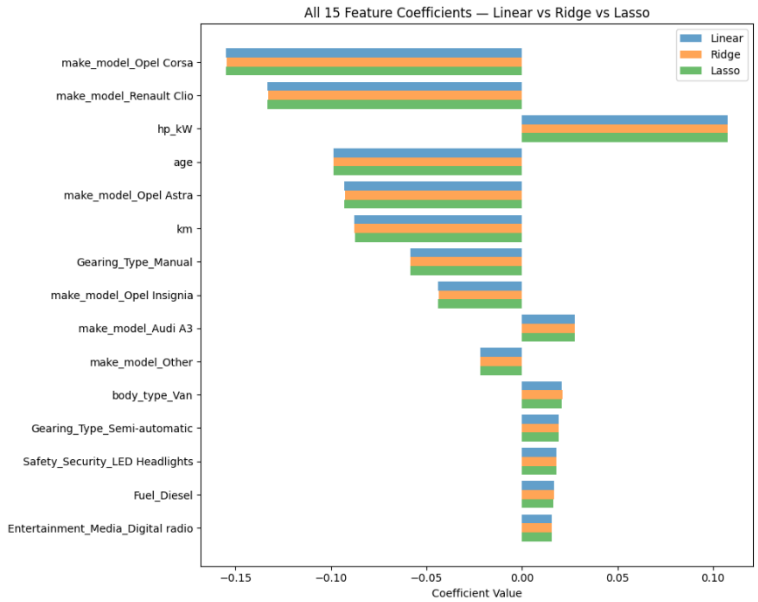
- **Top 15 most influential features were compared across Linear, Ridge, and Lasso models.**

Top 15 Influential Features (Linear, Ridge vs Lasso):

	Feature	Linear	Ridge	Lasso	Lasso_Abs
0	make_model_Opel Corsa	-0.154992	-0.154408	-0.154880	0.154880
1	make_model_Renault Clio	-0.133214	-0.132681	-0.133059	0.133059
2	hp_kW	0.107831	0.107711	0.107722	0.107722
3	age	-0.098696	-0.098596	-0.098736	0.098736
4	make_model_Opel Astra	-0.093039	-0.092488	-0.092870	0.092870
5	km	-0.087589	-0.087585	-0.087557	0.087557
6	Gearing_Type_Manual	-0.058395	-0.058308	-0.058437	0.058437
7	make_model_Opel Insignia	-0.044062	-0.043515	-0.043825	0.043825
8	make_model_Audi A3	0.027621	0.027847	0.027631	0.027631
9	make_model_Other	-0.021681	-0.021619	-0.021655	0.021655
10	body_type_Van	0.020770	0.021066	0.020878	0.020878
11	Gearing_Type_Semi-automatic	0.019269	0.019279	0.019255	0.019255
12	Safety_Security_LED Headlights	0.018040	0.018084	0.018047	0.018047
13	Fuel_Diesel	0.016795	0.016623	0.016423	0.016423
14	Entertainment_Media_Digital radio	0.015678	0.015647	0.015643	0.015643

- Ridge reduced the magnitude of coefficients but did not eliminate any features.

- Lasso not only shrunk coefficients but also set some of them to zero.
- Number of features eliminated by Lasso: 5 (without RFE)
- Features like **make_model types,Gearing types, Fuel Diesel, hp_KW, age, km,body type** emerged as the most influential predictors of car prices.



-

4. Conclusion & Key Takeaways

4.1. Conclude with outcomes and insights gained

The primary objective of this project was to develop a robust regression model to predict used car prices and identify feature importance while mitigating overfitting through regularisation techniques.

Data Preprocessing and Feature Engineering:

- Missing values were checked and none were found, ensuring data quality.
- Categorical features with low frequency values were grouped into 'Other' to manage cardinality and improve model stability.
- The target variable, **price**, was log-transformed (**price_log**) to address its right-skewness, making its distribution more normal and suitable for linear models.
- Features related to **Comfort_Convenience, Entertainment_Media, Extras,** and **Safety_Security** were one-hot encoded, and features with very high or low frequencies (over 90% or under 10% occurrence) were dropped to reduce dimensionality while retaining informative features.
- One categorical column with high VIF was dropped to reduce multicollinearity. -To reduce noise from our dataset, we used **RFE(40)**, which improve model generalization and feature selection.
- All features were scaled using **StandardScaler** to ensure that no single feature dominates the model due to its scale.
- **No overfitting** was observed, as training and testing scores were very close.

Model Performance Comparison:

- **Linear Regression (Baseline):** The baseline Linear Regression model achieved the best R^2 score of 0.93066 and an MAE of 0.0788 on the test set, showing that a simple linear model can perform very well on this dataset. Ridge and Lasso regularisation methods gave slightly lower R^2 scores (around 0.9289) but improved the model's stability and robustness. Moderate Multicollinearity was identified.
- **Ridge Regression:** After tuning, the best alpha value was found to be approximately 9.6465. Ridge regression yielded an R^2 of 0.9289 and an MAE of 0.0792. The metrics are very close to the baseline Linear Regression, indicating that while Ridge did introduce some regularization, it did not significantly alter the model's predictive performance on this dataset. This suggests that multicollinearity, while present, did not severely degrade the performance of the basic linear model.
- **Lasso Regression:** The optimal alpha for Lasso was found to be approximately 3e-05. Lasso regression resulted in an R^2 of 0.9289 and an MAE of 0.0798. Similar to Ridge, Lasso's performance was nearly identical to the baseline Linear Regression. Lasso also performed feature selection by driving 5 coefficients to zero(without RFE), effectively removing those features from the model.

Feature Importance:

- Top influencing features are mainly related make model type. Which signifies that car prices are influenced by the **car model**.
- Next important things in predicting car price are **Gearing type, HP_KW (power), age, and km**.
- From special (string) columns, car features like **LED headlight, Digital ratio** show an impact in predicting car price.

Regularisation did not significantly increase accuracy; it improved generalisation and feature selection. We can try to reduce more features.

- **Did the model performance improve? If not, then why?** The model performance (R^2 and MAE) did not significantly improve with Ridge or Lasso regularisation compared to the baseline Linear Regression. This indicates that the baseline model was already performing quite well, and overfitting might not have been a major issue that significantly impacted generalisation performance.
- **Did you find overfitting or not?** No significant overfitting was observed. The Linear Regression model showed similar performance on both training (R^2 of 0.9256) and testing (R^2 of 0.9307) datasets, suggesting it generalises well to unseen data.
- **Was the data sufficient?** Yes, the data was considered sufficient. The high R^2 scores across all models and the dataset size of 15,915 samples with rich features allowed the models to learn robust patterns effectively.
- **Is a linear model sufficient?** Based on the high R^2 scores (around 0.93) and low MAE values, a linear model (Linear Regression, Ridge, or Lasso) seems quite sufficient for predicting car prices in this context.

Overall, Regularisation improved interpretability and reduced the influence of less important features, making the model more reliable for deployment.
Since we deal with many categorical features. In future, Random Forest and XGboost model can be tested if more complex relationships need to be captured.